UNIVERSITY OF WOLVERHAMPTON

HERALD COLLEGE KATHMANDU

Decoding and encoding PNG files
Week 6

Prepared by: Uttam Acharya

UNIVERSITY OF WOLVERHAMPTON

HERALD COLLEGE KATHMANDU

Agenda

- Understanding how images are handled in C
- Working with PNG files using **lodepng**
- Pixel-level manipulation techniques
- Applications in graphics, vision, and data processing

UNIVERSITY OF WOLVERHAMPTON

HERALD COLLEGE KATHMANDU

Raster Images and PNG Format

- A digital image (raster image) is a 2D grid of pixels, which are the smallest individual units of a picture.
- Each pixel uses the **RGBA** model **(Red, Green, Blue for color and Alpha for transparency),** with values typically ranging from 0–255.
- PNG(Portable Network Graphics) is a lossless file format that stores pixel data and transparency information without loss of quality.

The ‘Encoded’ vs ‘Raw’ data

- Encoded data (like PNG files) uses compression and metadata to minimize file size, making it ideal for saving to a disk or transferring over networks.
- Raw data is an uncompressed array of pixel values (typically RGBA) stored in memory, providing the direct access needed for mathematical operations.
- Image manipulations, such as filtering or color adjustments, cannot be performed on encoded files; they require the "decoded" raw format to alter individual pixels.
- A typical image processing cycle involves **decoding a file into raw data, processing the pixel array, and re-encoding the result back into a compressed format** for storage.

UNIVERSITY OF WOLVERHAMPTON

HERALD COLLEGE KATHMANDU

Intro to loadpng_decode32_file

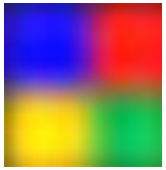
- Reads an encoded PNG from the disk and transforms it into a raw, uncompressed pixel array stored in the system's memory.
- Standardizes the output into a **32-bit RGBA** format, where every pixel is represented by 4 bytes (Red, Green, Blue, and Alpha components).
- It automatically identifies and retrieves the image's width and height, storing them in provided variables for use in processing loops.
- By providing direct access to the pixel values, it enables the implementation of various algorithms such as color transformations, spatial filtering, and geometric edits.

UNIVERSITY OF WOLVERHAMPTON

HERALD COLLEGE KATHMANDU

Pixel structure in a 4*4 image (RGBA)

Lets say we have the following 4x4 image (fairly small image):



The top left is made up of 2x2, R = 255, G = 36, B = 36, T = 255

The top right is made up of 2x2, R = 40, G = 23, B = 255,T = 255

The bottom left is made up of 2x2, R = 34, G = 177, B = 76, T = 255

The bottom right is made up of 2x2, R = 255, G = 203, B = 23, T = 255

Decoding image Using lodepng in C

Single Pixel Processing

- The lodepng library provides several functions for decoding PNG images. One of the simplest is:

```
lodepng_decode32_file(unsigned char** out, unsigned* w, unsigned* h,
  const char* filename)
```

- out **:** Pointer to a 1D array that will store the decoded image data (RGBA values)
- w and h : Returns the width and height of the image
- filename: Name/path of the PNG file to be loaded
- This function reads a PNG file and converts it into a raw pixel array (RGBA format) that can be processed in C/C++ programs.

Single Pixel Processing – decoding – code within the main part 1

```
unsigned int error;
unsigned char* image;
unsigned int width, height;

error = lodepng_decode32_file(&image, &width, &height,
filename);

if(error){
  printf("error %d: %s\n", error, lodepng_error_text(error));
}
```

Single Pixel Processing – decoding – code within the main part 1

```
printf("width = %d height = %d\n", width, height);

for(int i = 0; i<width*height*4; i=i+4){

printf("%d %d %d %d\n", image[i], image[1+i], image[2+i],
image[3+i]);

}
```

Single Pixel Processing decoding

- Notice that the array begins with the data from the top row. In this example, we get
  - blue, blue, red, red
  - blue, blue, red, red
  - yellow, yellow, green, green
  - yellow, yellow, green, green
- The transparency value for this example is 255 which means no part of the pixel is see through (opaque). Some images will have sections which will be completely transparent therefore this value will be 0.

Single pixel processing – encoding

The last part of any image processing is encoding the processed array into an image. Once the pixel data has been processed,

lodepng_encode32_file(const char* filename, const unsigned char* image, unsigned w, unsigned h)

This function requires the new name of the file you wish to generate (filename) and the array of numbers you pass in (image (RGBT values)).

Demo will be shown in the lecture on how to read in a PNG file, print the raw data and generate a file with a single dimension array.

UNIVERSITY OF WOLVERHAMPTON

HERALD COLLEGE KATHMANDU

Converting a single array image into a 2D array

Problems with single array image data

Single array images are great for single pixel manipulation. For example, if we have the following 2x2 single pixel array:

{237,28,36,255,34,177,76,255,255,242,0,255,237,28,36,255}

We can add some noise to the image by changing the values, lets add 50 to each COLOUR value depending on if the value is less than 205 (as max is 255). Our new image array will hold the following values:

{237,78,86,255,84,227,126,255,255,242,50,255,237,78,86,255}

UNIVERSITY OF WOLVERHAMPTON

HERALD COLLEGE KATHMANDU

Multi-pixel processing with 1D array (6x4)

237,28,36,255,237,28,36,255,34,177,76,255,34,177,76,255,255,242,0,255,255,242,
0,255,237,28,36,255,237,28,36,255,34,177,76,255,34,177,76,255,255,242,0,255,25
5,242,0,255,127,127,127,255,127,127,127,255,63,72,204,255,63,72,204,255,185,12
2,87,255,185,122,87,255,127,127,127,255,127,127,127,255,63,72,204,255,63,72,20
4,255,185,122,87,255,185,122,87,255

The above array represents a 6x4 image however, in single dimension,it's very hard to know which pixel is at which coordinate. Let's format the data into it's correct orientation

Single dimension to 2D array

Single array data holds pixels in sequence. It's not very easy to detect the surrounding pixels of 1 single pixel.

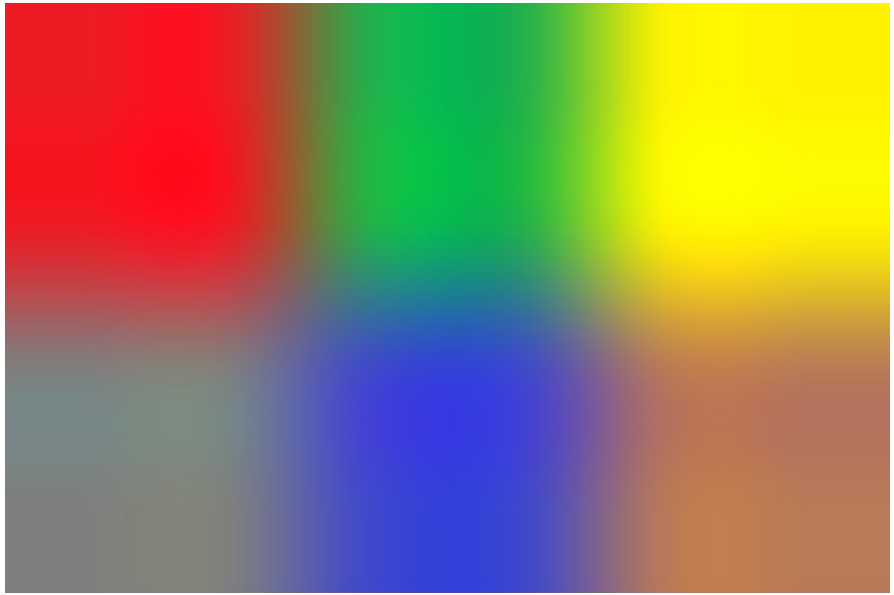
Inputs needed to create a 2D array:

1. Width of image
2. Height of image
3. Number of values per pixel (for this library, we use 4).

UNIVERSITY OF WOLVERHAMPTON

HERALD COLLEGE KATHMANDU

The 6x4 image



UNIVERSITY OF WOLVERHAMPTON

HERALD COLLEGE KATHMANDU

Single Pixel Processing – decoding – code within the main part 1

```
unsigned int error;

unsigned char* image;

unsigned int width, height;

const char* filename = argv[1];


error = lodepng_decode32_file(&image, &width, &height, filename);
if(error){
  printf("error %d: %s\n", error, lodepng_error_text(error));
}
```

Single Pixel Processing – decoding – code within the main part 1

```
printf("width = %d height = %d\n", width, height);

for(int i = 0; i<height*width*4 ; i=i+4){

printf("%d %d %d %d\n", image[i], image[1+i], image[2+i],
image[3+i]);

}
```

Main Code – creating 2D array variable in C

You can create a 2D array in C using the following syntax:

variabletype nameofvariable[rowsize][colsize];

For our program, the variable type is an "unsigned char," and the dimensions are height as our rows, and width*4 (4 values per pixel) for our columns.

unsigned char image2D[height][width*4];

Populating the 2D Array - Theory

We will take that horrible long list of numbers (in slide 15) and format it into a 6x4 matrix. This is simply 4 rows of data which holds 24 values. The 24 values is derived by the total of pixels in the “x” multiplied by 4 (RGBT).

The end product should look like the following:

237,28,36,255,237,28,36,255,34,177,76,255,34,177,76,255,255,242,0,255,255,242,0,255
237,28,36,255,237,28,36,255,34,177,76,255,34,177,76,255,255,242,0,255,255,242,0,255
127,127,127,255,127,127,127,255,63,72,204,255,63,72,204,255,185,122,87,255,185,122,87,255
127,127,127,255,127,127,127,255,63,72,204,255,63,72,204,255,185,122,87,255,185,122,87,255

Main code – populating 2D array

```
for(int row=0; row<height; row++){
        for(int col=0; col<width*4; col=col+4){
                image2D[row][col]=image[(row*width*4)+col];
                image2D[row][col+1]=image[(row*width*4)+col+1];
                image2D[row][col+2]=image[(row*width*4)+col+2];
                image2D[row][col+3]=image[(row*width*4)+col+3];}}
```

We use 2 nested for loops which act as coordinate counters for our 2D array.
We jump every 4 values for the width as each pixel holds 4 attributes.

Print contents of 2D array

```
for(int row1=0; row1<height; row1++){
        for(int col1=0; col1<width*4; col1++){
                printf("%d ", image2D[row1][col1]);
                if((col1+1)%4 == 0){
                        printf("| ");}}
        printf("\n");
}
```

UNIVERSITY OF WOLVERHAMPTON

HERALD COLLEGE KATHMANDU

Overview of image filters

- **Grayscale filter:**

  A grayscale filter transforms a PNG image from color into shades of gray by removing the color information and keeping only brightness (intensity).
- **Negative filter**

  A negative filter (invert filter) transforms an image by reversing all its colors, creating a photo-negative effect.

Grayscale filter:

Formula for grayscale filter

gray = 0.299R + 0.587G + 0.114B

Then

R = G = B = gray

Pixel transformation

Before(Color image): (R, G, B, A)

After (Grayscale image): (gray, gray, gray, A)

UNIVERSITY OF WOLVERHAMPTON

HERALD COLLEGE KATHMANDU

Grayscale filter on 1D array image

```
for (int i = 0; i < width * height * 4; i += 4) {

    unsigned char r = image[i + 0];
    unsigned char g = image[i + 1];
    unsigned char b = image[i + 2];

    // grayscale formula
    unsigned char gray = (unsigned char)(0.299 * r + 0.587 * g + 0.114 * b);

    image[i + 0] = gray; // R
    image[i + 1] = gray; // G
    image[i + 2] = gray; // B

    // Alpha (A) unchanged
}
```

UNIVERSITY OF WOLVERHAMPTON

HERALD COLLEGE KATHMANDU

Negative filter:

Formula for negative filter

For 8-bit color channels (0–255):

$R' = 255 - R, G' = 255 - G, B' = 255 - B$

Pixel transformation

Before(Color image): (R, G, B, A)

After (Negative Image): $(255 - R , 255 - G , 255 - B , A)$

UNIVERSITY OF WOLVERHAMPTON

HERALD COLLEGE KATHMANDU

Negative filter:

```
for (i = 0; i < width * height * 4; i += 4) {
    image[i + 0] = 255 - image[i + 0]; // R
    image[i + 1] = 255 - image[i + 1]; // G
    image[i + 2] = 255 - image[i + 2]; // B
    // A unchanged
}
```

UNIVERSITY OF WOLVERHAMPTON

HERALD COLLEGE KATHMANDU

Summary

- Decoding and encoding using LodePNG
  - 1D array image and 2D array image
- Working with some image filters
  - Grayscale and negative

UNIVERSITY OF WOLVERHAMPTON

HERALD COLLEGE KATHMANDU

Thank you…